

TradeOps sur CRC Platform Engineering Day-2 / N3 / RCA

Dossier de diagnostic top-down, lecture du mode PARK, gouvernance, reseau, securite, capacity management et preuves de runtime.

Dossier de démonstration et diagnostic technique

ENVIRONNEMENT

OpenShift Local / CRC 4.22.7, un noeud

COLLECTE

89 commandes read-only, 12 sections

DEPOT CIBLE

zdmoooc/TradeOps-GenAI-Integration

PREUVE

Archive UTC 20261008T162238Z

Date de capture runtime : 08/10/2026 à 16:22:38 UTC

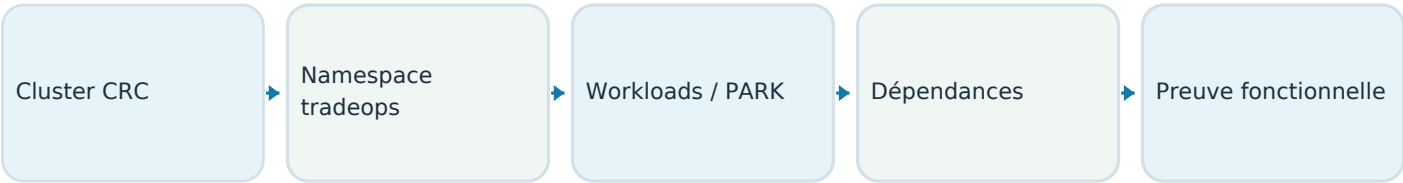
Zidane Djama | Architecte Solutions / Technique & Transverse

Perimetre : laboratoire local, sans preuve HA ni production.

01

Objet, portée et niveau de preuve

La santé CaaS, la disponibilité applicative et la preuve métier sont trois niveaux différents.



Classe	Définition / application
A - Observation	Capture read-only du 08/10, état actuel du cluster
B - Preuve antérieure	R5 MCP/IBM MQ en fenêtre bornée, conservée dans 12-evidence.txt
C - Déclaratif	Chart, documentation, runbooks et configuration des dépôts
D - Non prouvé	HA, production et scénario HITL complet sur cette capture

Lecture

Le dossier n’assimile pas Deployments à 0/0 à une panne. Les résultats observés sont qualifiés selon leur horodatage et leur source.

Source : 00-summary.txt ; 12-evidence.txt ; runbook PARK.

Architecture OpenShift : lecture par plans

Réutilisation de la méthode du dossier CaaS Shared, adaptée au produit TradeOps.



Plan	Questions d'exploitation
Contrôle	ClusterVersion, Operators, node, scheduler et conditions
Plateforme	OLM, GitOps, monitoring, image registry et Platform Operator
Application	Namespace tradeops, quotas, SCC, RBAC, NetworkPolicy
Fonctionnel	MCP, GenAI, workflows et systèmes externes

Lecture

Une Route admise confirme la configuration Ingress ; elle ne prouve pas la présence d'un Pod Ready ou un parcours métier fonctionnel.

Source : 01-cluster.txt ; 04-network.txt ; 07-operators.txt.

Méthode RCA top-down et contrôle des biais

Réduire le périmètre à partir du symptôme, conserver les échecs et ne pas inventer une cause.



Étape	Principe
Séparer	État actuel vs événements historiques et PARK volontaire
Corréler	Horodatage, revision d'image, phase Pod et effet métier
Conclure	Cause racine seulement si la preuve soutient le mécanisme
Valider	Non-régression documentée, pas de mutation sans approbation

Lecture

89 commandes EXIT_CODE=0 signifient collecte sans erreur de CLI, pas « 89 contrôles fonctionnels validés ».

Source : 00-summary.txt ; 11-rca.txt.

ClusterVersion et ClusterOperators

Instantané à 16:22 UTC le 08/10/2026.

4.22.7

Version OpenShift

25/25

Operators disponibles

Ready

Nœud CRC

Objet	Observation
ClusterVersion	Available=True, Progressing=False
ClusterOperators	25 Available=True ; Progressing=False ; Degraded=False
MachineConfigPool master	Updated=True ; Updating=False ; Degraded=False
MachineConfigPool worker	Aucun noeud worker séparé (0)

Lecture

Plateforme convergée lors de la capture. Le caractère mono-nœud ne démontre aucune tolérance à la perte du nœud.

COMMANDES DE DIAGNOSTIC / REPRODUCTION

```
$ oc get clusterversion version
$ oc get clusteroperators
$ oc get nodes -o wide
```

Source : 01-cluster.txt.

05

CRC : ressources du noeud et contraintes

Un seul noeud cumule control plane et workloads.

8 vCPU

Capacité physique annoncée

21 %

CPU instantané

80 %

Mémoire instantanée

Mesure	Observation
CPU allocatable	7800m ; CPU requests 7268m (93 %)
Mémoire allocatable	24143204Ki ; requests 23139Mi (98 %)
Mémoire mesurée	18926Mi (80 %) via oc adm top node
Conditions	MemoryPressure=False ; DiskPressure=False ; PIDPressure=False

Lecture

Risque prioritaire : très faible réserve de scheduling. Utilisation CPU faible ne signifie pas qu'un nouveau Pod peut être planifié.

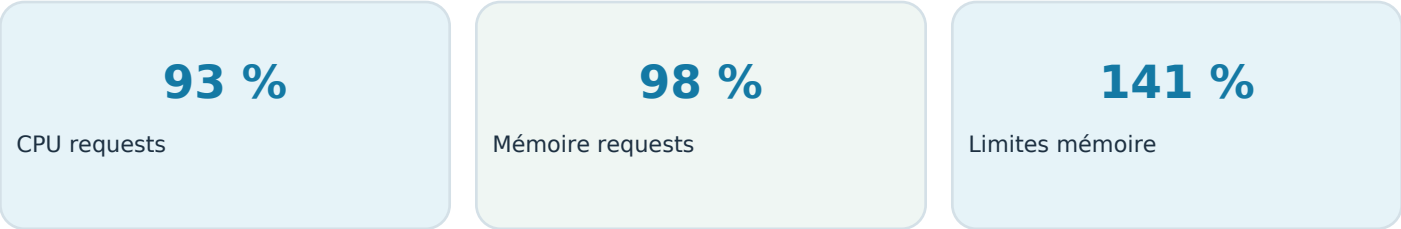
COMMANDES DE DIAGNOSTIC / REPRODUCTION

```
$ oc adm top nodes
$ oc describe node crc
$ oc get pods -A --field-selector=status.phase=Pending -o wide
```

Source : 02-capacity.txt.

CPU / RAM : requests, limits et consommation

Savoir distinguer la charge mesurée, les réservations du scheduler et le sur-engagement des limites.



Indicateur	Interprétation
Réservation CPU	7268m / 7800m ; marge ≈ 532m
Réservation RAM	23139Mi sur environ 23577Mi allocatable
Limites CPU	29910m (383 % allocatable)
Limites RAM	33258Mi (141 % allocatable)
Pods Pending	Aucun lors de la capture

Lecture

La marge RAM réservée est de quelques centaines de MiB, même si la mémoire consommée est plus basse. Prévoir les reprises en fenêtres bornées.

Source : 02-capacity.txt (describe node, top, Pending).

Namespace, ResourceQuota et LimitRange

Gouvernance des ressources du consommateur TradeOps.

15

Deployments

3

StatefulSets

0/8

PVC utilisés

Ressource	Observation
Quota CPU requests	225m / 4 coeurs
Quota mémoire requests	768Mi / 8Gi
Quota limits	CPU 1500m / 12 ; RAM 2560Mi / 20Gi
LimitRange	tradeops-defaults présent
Quota stockage	0 / 20Gi ; 0 PVC courant

Lecture

Le quota local ne remplace pas la contrainte du nœud CRC : l’allocatable global est presque entièrement réservé.

Source : 02-capacity.txt ; 05-security.txt.

PARK : état attendu des Deployments

Le PARK empêche de juger la santé fonctionnelle depuis une URL inactive.

14/15

Deployments à 0/0

1/15

Prometheus à 1/1

PASS

R5 re-PARK historique

Classe	Lecture
Arretés normalement	agent-controller, genai-api, workflow-api, tradeops-ui, risk-engine...
En activité	prometheus 1/1
MCP natif R5	mcp-native retiré après la fenêtre bornée (snapshot initial)
Restauration	R5_CRC_WINDOW_REPARK=PASS dans la preuve précédente

Lecture

PARK est un état d'exploitation économisant les ressources. Une Route/Service persistante peut pointer vers zéro EndpointSlice pendant PARK.

Source : 03-workloads.txt ; 04-network.txt ; 12-evidence.txt.

StatefulSets : préserver les données locales

PostgreSQL, Qdrant et Redpanda sont volontairement exclus du scale-to-zero normal.

1/1

PostgreSQL

1/1

Qdrant

1/1

Redpanda

Rôle	État actuel
PostgreSQL	1/1 Ready ; 22 MiB mesurés
Qdrant	1/1 Ready ; 61 MiB mesurés
Redpanda	1/1 Ready ; 178 MiB mesurés
Prometheus	Deployment 1/1 ; 32 MiB mesurés

Lecture

Le profil CRC documente emptyDir pour certains états ; ne pas assimiler Ready=1/1 à un contrat de persistance éprouvé.

Source : 03-workloads.txt ; 09-observability.txt ; docs/runbooks/CRC_TRADEOPS_PARK_RESUME.md.

10

Pods, ReplicaSets et traces d'exécution

Contrôleurs désirés et anciens Pods sont deux objets d'état distincts.

Objet	Qualification
Pod notifier ancien	ContainerStatusUnknown, 0/1, 12 redémarrages, âge 26 jours
Build runtime-12 ancien	ContainerStatusUnknown ; Build historique Failed
Build UI-2 ancien	Pod Error ; Build historique DockerBuildFailed
Build runtime-16	Build Complete ; image courante publiée
Pods actifs data	Postgres, Qdrant, Redpanda et Prometheus Running 1/1

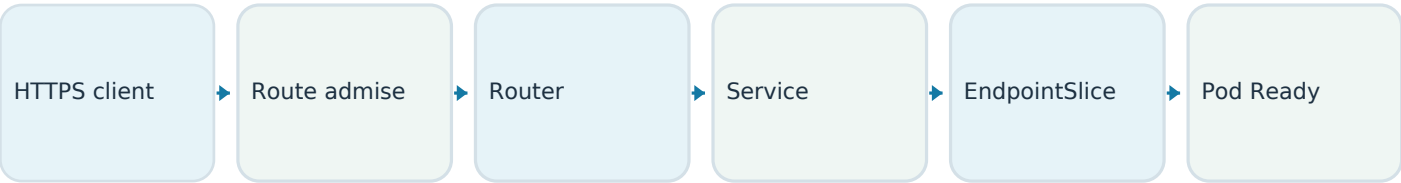
Lecture

Objets historiques résiduels : investiguer owner, âge et référence à une preuve avant nettoyage ; aucun delete automatique.

Source : 03-workloads.txt ; 10-dependencies.txt.

Route → Service → EndpointSlice → Pod

Le parcours HTTP doit être validé saut par saut.



Niveau	Preuve
Routes	4 : agent-controller, grafana, tradeops-ui, workflow-api
Admitted=True	Les quatre Routes observées
EndpointSlice en PARK	Absent pour les Deployments stoppés (agent-controller, UI...)
EndpointSlice data	Postgres, Qdrant, Redpanda et Prometheus présents

Lecture

Routes admises mais pas d’endpoint applicatif pendant PARK : configuration saine, disponibilité applicative volontairement suspendue.

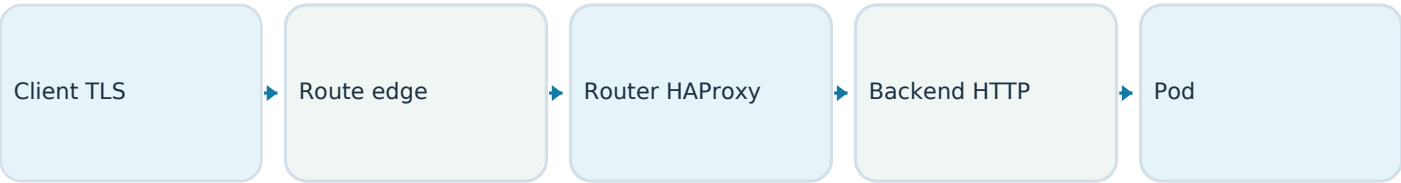
COMMANDES DE DIAGNOSTIC / REPRODUCTION

```
$ oc -n tradeops get routes -o wide
$ oc -n tradeops get endpointslices.discovery.k8s.io -o wide
```

Source : 04-network.txt.

Ingress OpenShift et terminaison TLS

Séparer le niveau TLS de l'état applicatif.



Mode	Fait / limite
TradeOps (4 Routes)	Termination=edge ; Redirect ; Admitted=True
Reencrypt	Motif OpenShift possible, non utilisé par ces 4 Routes TradeOps
Passthrough	Motif OpenShift possible, non utilisé par ces 4 Routes TradeOps
Vérification restante	Headers et réponse HTTP seulement en fenêtre ACTIVE approuvée

Lecture

Une Route TLS correctement admise ne rend pas le service opérationnel si le Deployment sous-jacent est à 0 replica.

Source : 04-network.txt.

Services applicatifs et câblage de données

Les noms Service persistent même lorsque les Pods ont été parqués.

Composant	État réseau à la capture
agent-controller	Service 8015 ; aucun endpoint en PARK
genai-api	Service 8013 ; aucun endpoint en PARK
mcp-server	Service 8016 ; aucun endpoint en PARK
postgres	Service 5432 ; endpoint présent
redpanda	Service 9092 et 33145 ; endpoint présent
qdrant	Service 6333 ; endpoint présent

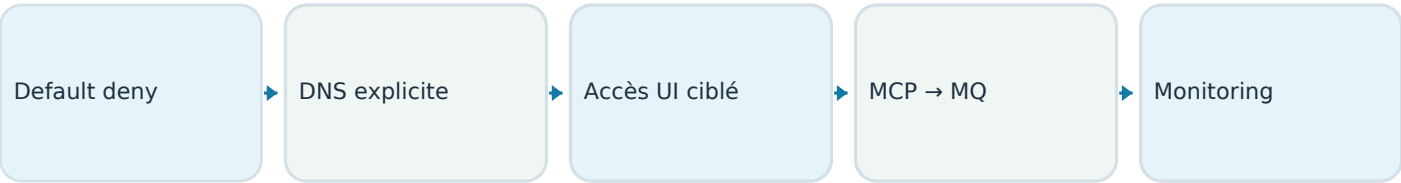
Lecture

Le namespace compte des Services configurés ; l'existence d'un Service ClusterIP ne prouve pas l'accessibilité d'un backend.

Source : 04-network.txt.

NetworkPolicy : séparation des flux

La sécurité Est/Ouest combine default-deny et autorisations nommées.



Contrôle	Configuration observée
default-deny	NetworkPolicy présente
Flux intra namespace	allow-intra-namespace, sauf tradeops-ui
MCP vers IBM MQ	allow-mcp-native-to-mayabank-mq-ops
Identité / IA	Flux Keycloak, OTel, Kong et Ollama explicités
Ingress externe	allow-router-ingress sur les Routes concernées

Lecture

La preuve négative R5 a vérifié le bypass direct refusé ; la capture de configuration seule ne prouve pas toutes les combinaisons de flux.

Source : 04-network.txt ; 12-evidence.txt.

SCC, identités et RBAC

Distinguer l'identité du workload de celle d'administration et des builds.

Contrôle	Observation
Pods TradeOps Running	SCC restricted-v2 : Postgres, Qdrant, Redpanda, Prometheus
Ancien notifier	SCC restricted-v2 (Pod ancien)
Build pods historiques	SCC privileged et SA builder : contexte build, non workload
ServiceAccounts	tradeops-runtime, default, builder, etc.
Compte de collecte	kubeadmin : can-i get/create = yes ; ne mesure pas les droits SA

Lecture

L'accès kubeadmin ne démontre pas le moindre privilège de TradeOps. Prévoir tests RBAC sous l'identité exacte en fenêtre de qualification.

Source : 05-security.txt.

OLM : Operator Lifecycle Manager

La chaîne CatalogSource → Subscription → InstallPlan → CSV → Operator doit être suivie.



Composant	Observation
OpenShift GitOps	CSV v1.22.1, Succeeded
OpenShift Pipelines	CSV v1.23.2, Succeeded
RHBK Operator	CSV v26.6.7-opr.1, Succeeded
Point de vigilance	CSV Succeeded ne garantit pas Ready=True d'une Custom Resource

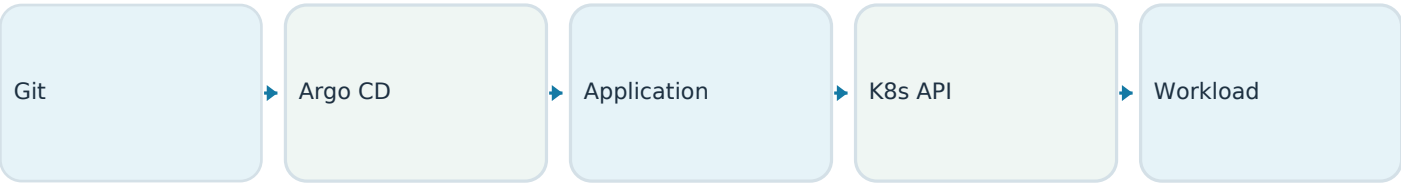
Lecture

Le Platform Operator MayaBank est distinct de la chaîne OLM Red Hat ci-dessus. Sa CR métier doit être inspectée indépendamment.

Source : 07-operators.txt.

GitOps Argo CD : état désiré et owner

La santé d’OpenShift GitOps ne démontre pas la convergence du produit TradeOps.



État	Interprétation
Applications Synced/Healthy	instant-payments-tech-lead-shared-platform, mayainsurance-decision
Applications Unknown/Healthy	mayabank-mq-local-core, mayabank-mq-local-db, wero-poc-crc
Application dédiée TradeOps	Aucune dans la liste de 5 Apps observées
Workload TradeOps	Labels Helm dominants ; le PARK protège contre conflit self-heal

Lecture

Unknown/Healthy exige lecture Git source/sync ; ne pas qualifier Synced ni OutOfSync sans statut fiable.

Source : 06-gitops.txt.

Platform Operator et CapabilityConsumption

Le mode Observe est une frontière d’ownership, pas un défaut à effacer.



Objet	État à qualifier
mayabank-platform-operator	Deployment 1/1 Running
tradeops-crc	Observe ; Ready=False (capture)
instant-payments-crc	Manage ; Ready=False (capture)
Adoption TradeOps	Manage non autorisé ; inventaire brownfield préalable

Lecture

Ready=False doit être rapproché des Conditions détaillées et de l’OwnershipConflict attendu. La capture n’inclut pas ce détail ; aucune cause non observée n’est affirmée.

Source : 07-operators.txt ; dossier D-093 précédemment versionné.

Stockage CRC : PVC absents et PV Released

Un StatefulSet Ready n'est pas une preuve de durabilité du stockage.



Artefact	Interprétation
data-postgres-0	PV Released historique, politique Retain
data-redpanda-0	PV Released historique, politique Retain
data-qdrant-0	PV Released historique, politique Retain
CRC runtime actuel	Volume stateful documented as emptyDir in CRC profile

Lecture

Ne pas supprimer les PV Released avant vérification de la provenance des données ; une suppression de Pod emptyDir peut entraîner une perte.

Source : 08-storage.txt ; CRC_TRADEOPS_PARK_RESUME.md.

Observabilité : probes, metrics et limites

Différencier stack installée, cible active et télémétrie démontrée.

Niveau	État
OpenShift monitoring	Pods principaux Running (prometheus-k8s, Thanos, etc.)
Prometheus TradeOps	Deployment 1/1, consommation 32Mi à la capture
ServiceMonitor	ai-access-policy présent ; PodMonitor absent dans tradeops
PrometheusRule	Aucune dans le namespace TradeOps
Preuve G2 antérieure	Prometheus + Shared OTel validés en fenêtre bornée, pas mesurés par cette capture

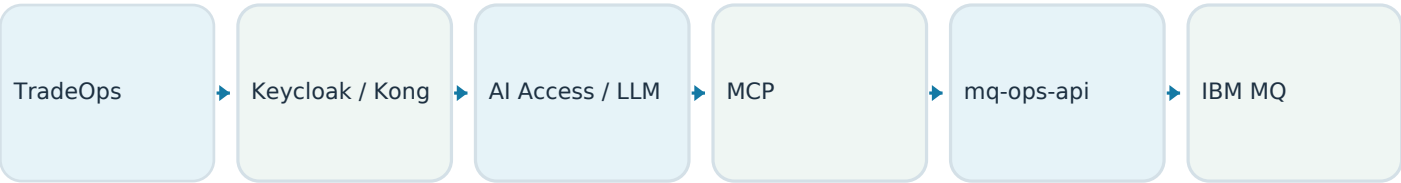
Lecture

L’inventaire ne contient ni requête PromQL, ni trace OTLP courante, ni accès API Grafana pendant PARK. Ne pas annoncer une couverture d’alerting complète.

Source : 09-observability.txt ; README et preuves D-090 G2.

Dépendances externes du produit

Les couches partagées et MayaBank MQ peuvent rester actives pendant PARK.



Namespace	État observé
mayabank-mq-local	MQ, mq-ops-api, exporter, payment-processing, payments-db : 1/1
mayabank-api	api-gateway et payment-api : 1/1
keycloak-system	Keycloak, PostgreSQL, rhbk-operator Pods Running
shared-platform-services	Platform Operator 1/1

Lecture

Ces dépendances présentes n’impliquent pas que TradeOps complet est actif ; elles sont réutilisées par des tests ponctuels.

Source : 10-dependencies.txt.

BuildConfig, images et traçabilité

L'image déployée R5 est liée à un commit explicite.

33a1faf

Source build #16

Complete

Build runtime #16

21f6984

Commit docs local

Élément	Preuve
BuildConfig tradeops-runtime	Git main, latest 16
Build runtime-16	Complete, environ 31m21s
Image tag i9	Digest sha256:5a174f53...
Historique build	runtime-12 Failed, runtime-13/14/15 Complete
UI build	ui-2 Failed historiquement, ui-4 Complete

Lecture

Une image publiée ne garantit pas que tous les Deployments en PARK pointent déjà vers ce digest. R5 a épinglé l'image lors de sa fenêtre bornée.

Source : 03-workloads.txt ; 10-dependencies.txt ; 12-evidence.txt.

Événements historiques et causalité

Les Warning peuvent appartenir à des tentatives précédentes pourtant réparées.

Signal	Lecture chronologique
trading-perso	FailedCreate MCP dû à SA tradeops-runtime absente ; erreur de namespace anciennement corrigée
tradeops/mcp-native	Startup probe refused pendant démarrages/transitions R5
tradeops/agent-controller	Readiness / startup refused pendant révisions R5
Validation ultérieure	R5_CRC_MCP_MQ_VERIFY_PASS puis re-PARK=PASS
Nettoyage	Pas de delete automatique, même si Pod ancien Unknown

Lecture

Un Warning historique après un redéploiement réussi n'est pas, à lui seul, un incident actif ; utiliser timestamp, owner et preuve fonctionnelle finale.

Source : 11-rca.txt ; 12-evidence.txt.

RCA 1 : mauvais namespace de déploiement

Exemple concret d'un diagnostic N3 appuyé par Events et correctif versionné.



Étape RCA	Constat
Symptôme	FailedCreate mcp-native dans trading-perso
Mécanisme	Contexte courant pointant sur trading-perso ; apply sans -n
Correction	Script R5 corrigé : oc -n "\$NAMESPACE" apply
Preuve après correction	R5 a atteint MCP_RUNTIME_PROVEN, puis re-PARK PASS

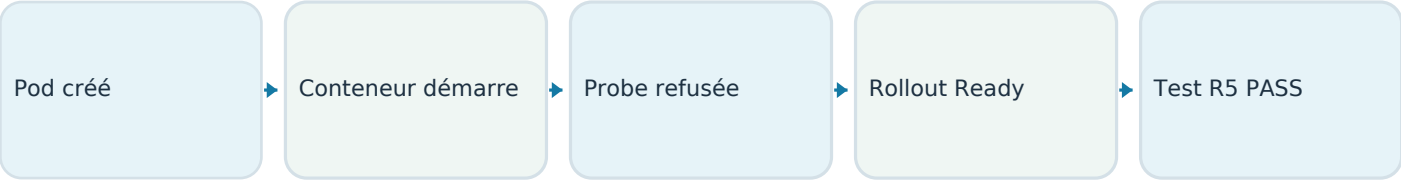
Lecture

Prévention : expliciter -n sur chaque mutation autorisée et vérifier le namespace rendu par Helm avant apply.

Source : 11-rca.txt ; correction PR #21.

RCA 2 : probes transitoires et image

Un probe refusé lors du startup n'est pas une panne persistante sans autre preuve.



Point	Preuve / limite
Observation	Unhealthy (startup/readiness TCP/HTTP) pendant fenêtres de test
Contexte	Démarrage de mcp-native / agent-controller, changements de ReplicaSets
Test concluant	MCP_TOOLS_PASS ; MQ_HEALTH_PAYLOAD_PASS ; HTTP 403
Délimitation	La capture PARK actuelle ne permet pas un smoke HTTP du service

Lecture

Pour une nouvelle panne active, relire les logs du Pod vivant en fenêtre autorisée ; ne pas conclure à une régression depuis des événements anciens.

Source : 11-rca.txt ; 12-evidence.txt.

RCA 3 : Pods Unknown et PV Released

Objets résiduels et persistance sont des risques de housekeeping non destructif.

Signal	Recommandation N3
Pod notifier inconnu	Owner, référence ReplicaSet, deletionTimestamp à confirmer
Build runtime-12 inconnu	Vérifier BuildConfig/build status ; dernier build sain
PV Released x3	Confirmer revendications précédentes, conservation et absence de données utiles
Nettoyage	Plan séquencé séparé, preuve avant/après et approbation

Lecture

Ne jamais faire `oc delete pod/pv` pour « obtenir du vert » avant de qualifier les conséquences storage et audit.

Source : 03-workloads.txt ; 08-storage.txt ; 10-dependencies.txt.

Registre des risques et actions Day-2

Priorisation basée sur l’impact potentiel, non sur le volume d’événements.

Priorité	Décision préconisée
P1 / Capacity	Requests CPU 93 % et RAM 98 % : capacité de reprise limitée
P1 / Data	emptyDir CRC / 0 PVC actifs ; protéger StatefulSets lors des manipulations
P2 / Sécurité	D-093 Observe Ready=False : clarifier Conditions et ownership
P2 / Hygiène	Pods Unknown et trois PV Released : inventaire contrôlé
P2 / Réseau	Routes admises sans endpoints pendant PARK : documenter accessibilité

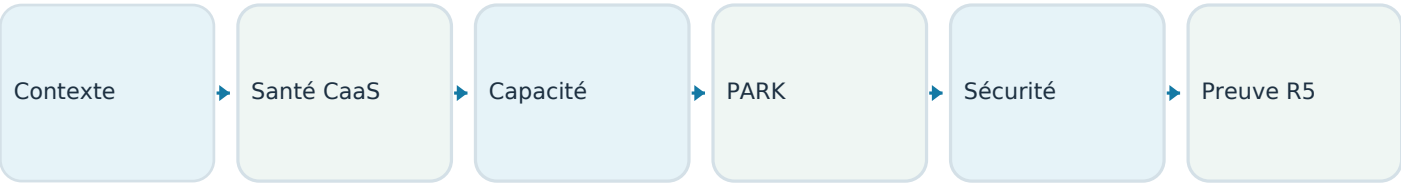
Lecture

La campagne de diagnostic est strictement read-only. Une action corrective sur CRC constitue une nouvelle étape avec fenêtre et rollback.

Source : Synthèse transversale 01 à 12.

Runbook de référence et conclusion

Parcours démonstratif 12 à 15 minutes : cluster → TradeOps → réseau → sécurité → preuve MCP-R5.



Gate	Verdict
G1 Santé du cluster	PASS : 4.22.7, 25 CO disponibles, node Ready
G2 Exploitation TradeOps	PARK observé ; data 3/3 + Prometheus actifs
G3 Risques	Capacity scheduling et stockage éphémère à qualifier
G4 MCP-R5	CRC_RUNTIME_PROVEN via preuve de fenêtre bornée
G5 Limite de conclusion	Ni HA ni production ; autres gates fonctionnels non déduits

Lecture

Restituer le diagramme, exposer 3 risques et illustrer 1 RCA ; ne pas réveiller le cluster pour la démonstration documentaire.

COMMANDES DE DIAGNOSTIC / REPRODUCTION

```
$ oc get clusterversion version && oc get clusteroperators
$ oc adm top nodes && oc -n tradeops get deploy,sts,pod
$ oc -n tradeops get route,service,endpointslice
$ oc get capabilityconsumptions -A
```

Source : Dossier CaaS Shared adapté ; collecte complète 20261008T162238Z.